

Do not remove nor postpone `access_context::unchecked`

Abstract

In [P3547](#), Dan Katz and I propose adding an `access_context`, and that has three different forms:

- `access_context::current()`, the current context's access
- `access_context::unprivileged()`, the equivalent of public access
- `access_context::unchecked()`, which bypasses access controls

SG7 discussed the paper, and had this poll result:

```
P3547R0: SG7 wants reflection to provide user-accessible access_context::unchecked() in C++26.
SF      F      N      A      SA
4       3       3       1       3

Result: not consensus
```

We should keep `access_context::unchecked`. It's necessary, it's necessary for generic algorithms, and it's part of a set of fundamental use cases of reflection.

Elaboration

Reflection (and injection) isn't just about auto-generating boiler-plate that you can already write manually. A significant part of it aims to make C++ a *programmable programming language*. And that aim means making C++ a programming language programmable in itself.

So what does that mean? Well, for the parts relevant for this discussion, it means that the language should expose its internal magic, the tricks that compilers know how to do, and allow reflection+injection metaprograms to use and reuse that magic.

It means that we absolutely should be able to write new traits with just reflection facilities. We shouldn't need to crack open a compiler's codebase and extend that to do so. We should be able to write new language facilities in terms of reflection, and if those facilities require probing members of a class, regardless of access, and looking at whether any of them have a particular property, we should be able to do so.

Because the point of a programmable programming language is that you can extend it without having to hack on a compiler.

Sometimes a new facility like that, written with reflection, needs to access otherwise inaccessible members, and needs to do so generically, without asking for a concrete type to cooperate. And it's infeasible to expect all types compatible with such a facility to somehow opt in to something, not all facilities are like that.

`access_context::unchecked` makes that possible. It also makes it visible, and easy to find, and easy to control and check that when you do things like that, they're subject to necessary scrutiny by reviewers, not hidden behind hacks. But nevertheless they are possible, like they need to be if we want to make the language more programmable and want to make it less necessary to change a compiler to extend the language. Like we have already agreed to do for reflection and injection, in many ways.